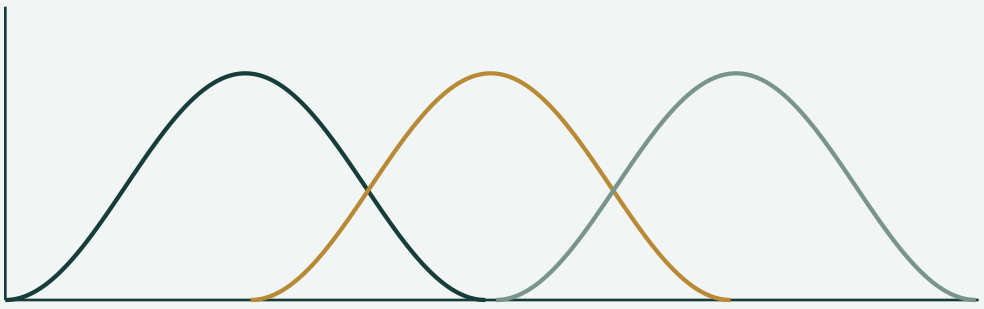
PARTICELLE

User Guide



time: overlapping envelopes sum at the output

Granular sound design, spatial composition, and expressive control

Practical workflows / Illustrated concepts
Listening exercises / Command and patch reference

Colby Leider, PhD

SOURCE-TREE EDITION / OCTOBER 10, 2026

Contents

Preface 3

How to use this book 4

1. Meet the instrument 5

2. Installation and a reliable workspace 8

3. Your first finished render 10

4. How grains become texture 12

5. Dronoify: a source becomes an atmosphere 14

6. Signals, curves, and controlled movement 16

7. Let the audio control the patch 19

8. Compose beyond stereo 21

9. Tuning and pitch: use the connected path 24

10. Realtime, MIDI, and OSC 26

11. Obelisk Mo for performers and host developers 29

12. A practical sound-design cookbook 31

13. Windows and the shape of a grain 33

14. Rendering, quality control, and archiving 35

15. Troubleshooting by symptom 37

16. Command and patch reference 39

17. A short course in granular composition 42

18. Glossary and further reading 44

Granular sound design, spatial composition, and expressive control

Book edition for the 0.1.0 source tree. Prepared October 10, 2026.

Preface

A recorded sound contains more than its original performance. A single note contains an attack, a changing spectrum, a steady region, and a decay. A field recording contains events at many different scales. Granular synthesis makes those regions available as material: extract short pieces, shape their edges, repeat them, overlap them, and distribute them in space.

Particelle describes that process in a text file. A patch is a recipe that can be read, revised, archived, and rendered again. This book teaches a working method for that instrument, from the first audible file to layered drones, automated studies, multichannel delivery, and MIDI or network control.

The book follows the teaching approach of the VerbX user guide: practical chapters first, illustrated explanations alongside the work, listening exercises, and a reference section for returning users. Read Chapters 1 through 4 in sequence. After that, choose a route: Chapter 5 for drones, Chapters 6 and 7 for automation, Chapters 8 and 9 for spatial work and tuning, or Chapters 10 and 11 for performance and Obelisk M0.

This edition was checked against the repository source, rather than treating every historical example or CLI help sentence as a guarantee. Particelle is alpha software. Several schema fields describe planned or library-level capabilities that the granular command-line path does not yet apply. The capability table in Chapter 1 and the notes beside the affected workflows explain those boundaries.

All listening tasks assume modest monitoring levels. Begin with short previews, examine the rendered level, and only then commit to a long render. A dense cloud can sum to a much larger level than its individual grains suggest.

How to use this book

Commands in `sh` blocks belong in a terminal. YAML and JSON blocks belong in files. Never paste a bare YAML patch into zsh or bash. The shell interprets braces, colons, and quotes as shell syntax; it does not become a patch editor because a block looks like configuration.

Full teaching patches and their curve are supplied in `examples/userguide/`. Work from the repository root so their relative paths resolve. The PDF and this Markdown source contain the same teaching text; the build script copies the named example blocks into the accompanying files.

Commands use `particelle` after installation. When working from source without installing, replace it with `./target/debug/particelle` after `cargo build -p particelle-cli`. Output examples use a directory named `renders`; create it first with `mkdir -p renders`.

The unit conventions are worth learning before touching a parameter: `duration` and `position` are seconds, `density` is grains per second, oscillator rates are hertz, and angles ending in `_deg` or used for orientation are degrees. A position of `0.5` means approximately half a second into a matching-rate source; it does not mean the middle of the file.

1. Meet the instrument

A patch is a score for small sounds

The granular engine reads a WAV file into memory and emits windowed excerpts called grains. A cloud is a family of grains sharing a source and a collection of parameter signals. Several clouds can run together: one may hold a stable tone, another scan a noisy edge, and a third create intermittent detail.

The layout declares the output channels and their directions. The engine distributes each grain according to the cloud's spatial settings. The render command writes those channels to a WAV file. The run command sends them to an audio device.

A useful working sequence is source, patch, validation, preview, revision, delivery. Validation catches some structural errors; a preview proves that the complete audio path can load the assets and produce the intended result. Neither step replaces listening.

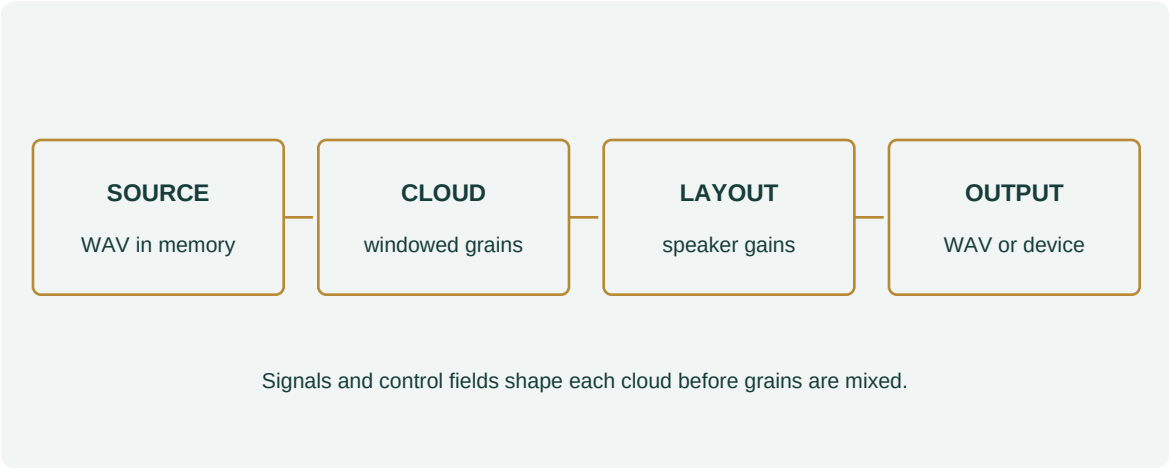


Figure 1. The granular signal path. A patch coordinates source selection, grain shape, and output geometry.

Choose the right entry point

Task	Entry point	Result
Make a first granular texture	`particelle init`	Editable two-cloud YAML patch
Turn a WAV into a drone	`particelle preset dronoify`	Three-cloud YAML patch
Produce an audio asset	`particelle render`	Multichannel WAV
Perform a granular patch	`particelle run`	Hardware output with live controls
Play an MPE synthesizer	`obelisk-m0-live`	Separate wavetable/modal instrument
Embed the Mo instrument	Rust API or C ABI	Host-controlled synthesis instance

Obelisk Mo belongs to the repository but is a separate instrument. It synthesizes voices from wavetable excitation and a resonant body. It does not consume a Particelle granular YAML patch, and its JSON controls are not interchangeable with the Dronoify generator.

What this edition supports

Area	Current user-facing behavior
Granular source	WAV file loaded into memory; source channels averaged inside each grain
Output format	32-bit float by default; PCM24 and PCM16 available
Patch generation	1 through 256 channels; counts above two generate a horizontal ring
Spatial rendering	Default CLI uses amplitude panning; channel names do not create an Atmos deliverable
Granular tuning	Tuning declarations validate, but CLI cloud playback rate is not connected to them
MPE tuning	Scala/KBM and host-supplied frequency tables reach the synthesizer
Realtime	Device-dependent; qualify the actual interface and patch
Native plug-ins	MPE adapter and C API exist; this tree does not ship a completed VST3/AU wrapper

These distinctions matter musically. An eight-channel ring is a useful compositional space, but it is not automatically a cinema bed. A tuning declaration can be valid YAML without changing granular pitch. A successful offline render does not demonstrate that a laptop can sustain the same patch in realtime.

Listening exercise

Choose one source you know well. Write down three qualities you want to preserve and three you want to transform. Examples include the consonants of a voice, a bowed attack, a periodic pulse, a noisy tail, or the location of an event. Keep this list beside your first patch; it is a more useful target than simply making the texture denser.

2. Installation and a reliable workspace

The source-tree route used in this book

The examples use repository assets, so a source checkout provides the most reproducible starting point. Use an existing checkout if you already have one; otherwise clone the repository and build the CLI.

```
git clone https://github.com/TheColby/Particelle.git
cd Particelle
cargo build -p particelle-cli
./target/debug/particelle --version
mkdir -p renders
```

Use a current stable Rust toolchain and the platform's native compiler tools. Hardware audio builds may also require operating-system audio development libraries. If your immediate goal is offline rendering, the CLI can be built without realtime features:

```
cargo build -p particelle-cli --no-default-features
```

That build supports offline commands. Build with the default features when you need hardware output. The application version alone does not identify every development change; record `git rev-parse HEAD` when comparing results from source builds.

Installed and packaged builds

The repository provides `install.sh` with stable, nightly, and source channels. From a checkout, the stable installation command is:

```
./install.sh --channel stable
particelle --version
```

Stable selects a tagged release, nightly selects a development build, and source builds from source. A release may lag behind this book's source-tree features. Consult `docs/RELEASE_CHANNELS.md` for version pinning and signature verification. The Homebrew formula is maintained at `Formula/particelle.rb`; use the repository installation instructions appropriate to the Homebrew version you have installed.

Keep the executable and the assets conceptually separate. Installing a binary does not mean that the shell's current directory contains the source WAV requested by a starter patch. If `audio/music_example.wav` is missing, work from the checkout and prepare the canonical samples, or edit the patch to point to your own WAV.

```
./scripts/generate_sample_pack.sh
```

That script requires SoX. The checked-in source tree may already contain the required material; avoid regenerating it merely to resolve a wrong working directory.

Paths that remain understandable

This CLI resolves source WAVs, analysis WAVs, and curve files relative to the process working directory. It does not consistently rebase them on the location of the patch. A patch stored under `examples/userguide/` can therefore refer to `audio/music_example.wav` when launched from the repository root.

For a personal project, keep one project root containing `patches/`, `audio/`, `curves/`, and `renders/`. Launch commands from that root. Use absolute paths only when you deliberately want to refer outside the project. Quote shell paths containing spaces.

Do not change `engine.sample_rate` casually when importing a source recorded at another rate. The current granular CLI loads source samples directly without inserting sample-rate conversion. Matching the patch rate to the source avoids unintended changes in playback speed and position interpretation.

Exercise: make a portable project

Create a new folder with a short WAV, a patch, and a one-line text note recording the command used to render it. Move the entire folder, change into its new root, and render again. If it fails, inspect relative paths before changing synthesis settings. Portability is a compositional convenience as well as a reproducibility tool.

3. Your first finished render

Start with the designed demonstration

The starter generator creates two animated clouds. Generate the patch, validate it, and render a ten-second PCM24 preview. The commands below contain no shell prompt or illustrative output lines, so they can be entered directly.

```
particelle init > first.yaml
particelle validate first.yaml
particelle render first.yaml -o renders/first.wav \
  --duration 10 --format pcm24
```

On macOS, `afplay renders/first.wav` plays a stereo file. On systems with SoX, `play renders/first.wav` is another option. You can also import the WAV into a DAW. Start with stereo even if your eventual destination has many speakers; the first task is to establish that the source, patch, and render command work together.

Listen for two overlapping textures rather than a dry replay of the source. Their positions and timing differ, so the material should change over time. If it is silent, check that the referenced source is audible before raising amplitude.

Inspect a small patch

The following complete patch is supplied as `examples/userguide/basic.yaml`. It is deliberately simpler than the generated demonstration. One cloud gives each control a clear audible role.

examples/userguide/basic.yaml

```
schema_version: 2
engine:
  sample_rate: 48000
  block_size: 256
layout:
  channels:
    - { name: L, azimuth_deg: -30.0 }
    - { name: R, azimuth_deg: 30.0 }
clouds:
  - id: study
    source: audio/music_example.wav
    density: 12.0
    duration: 0.08
    position: 0.5
    amplitude: 0.12
    width: 0.5
    listener_pos: { x: 0.0, y: 1.0, z: 0.0 }
    window: { type: hann }

particelle validate examples/userguide/basic.yaml
particelle render examples/userguide/basic.yaml \
  -o renders/basic.wav --duration 5 --format pcm24
```

The engine rate is 48,000 frames per second. A block contains 256 frames. The layout contains two output channels in the order L, R. The cloud emits roughly twelve grains per second; each grain lasts 80 milliseconds and begins near half a second into the source. The Hann window softens the grain's beginning and end.

Know what success means

At 48 kHz, a ten-second render contains 480,000 frames. Stereo contains two samples per frame, so it contains 960,000 scalar samples. Frame count and sample count are different quantities. A report of 480,000 frames for ten seconds of stereo is correct.

A non-empty WAV is not sufficient evidence of a successful musical result. Confirm that it has the intended rate, channel count, duration, and nonzero audio. Listen at the beginning, middle, and end. For a reusable sound, also listen through the loop seam or edit boundary you intend to use.

Pipes and stdin

A dash in the patch argument means read YAML from standard input. This lets a generator feed a renderer without an intermediate file:

```
particelle init | particelle render - \
  -o renders/piped.wav --duration 5 --format pcm24
```

Save the patch for any result you want to keep. A pipe is convenient for a quick demonstration, but a named file is easier to revise and archive. Avoid editing a generated multi-line signal expression with a simple line-oriented substitution; it can leave the old expression body behind.

Exercise: three versions, one change

Copy the basic patch three times. Change only density to 6, 12, and 24 grains per second. Render each for five seconds and compare them at matched playback level. Describe the transition from identifiable fragments to a more continuous texture. Keep duration unchanged so the comparison has one independent variable.

4. How grains become texture

Density and duration work together

Density controls how often grains begin. Duration controls how long they remain active. Their product is a useful estimate of simultaneous overlap: a cloud at 40 grains per second with 0.2-second grains has an average overlap near eight. This estimate is a design aid, not an exact promise about every block of the current scheduler.

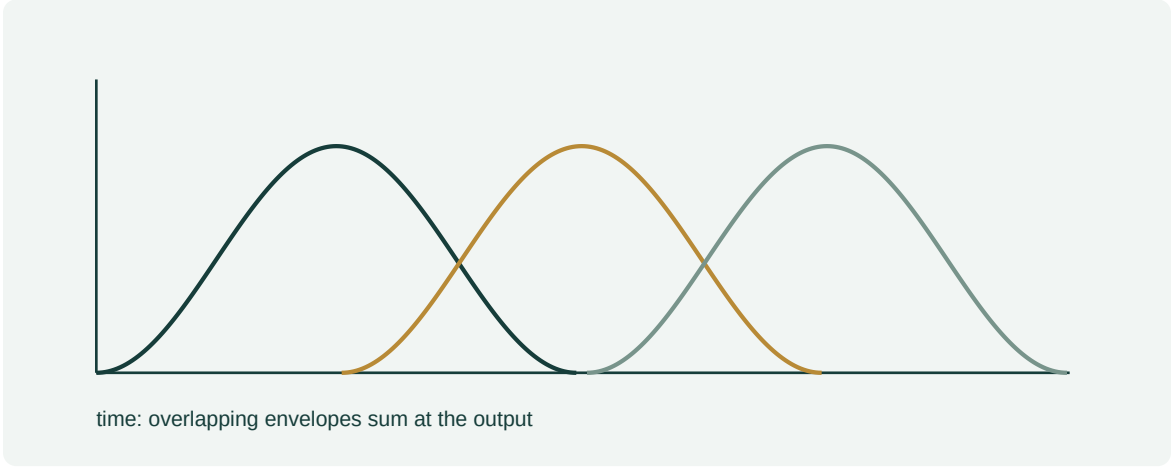


Figure 2. Three overlapping Hann envelopes. This schematic shows overlap, not a capture of scheduler timing.

Low overlap exposes individual grains and the gaps between them. High overlap blends details into a sustained texture. The source itself still matters: a voiced vowel may remain pitched, a cymbal may become a bright wash, and a drum transient may turn into a series of little impacts.

Increasing density and duration simultaneously changes the texture, level, and processing load. If you want only a smoother texture, start by increasing one of them and reducing amplitude. Compare renders after loudness matching so the louder version does not automatically seem better.

Source position is a reading location

A cloud's `position` selects where each new grain begins reading. A constant position revisits the same region. A slowly moving position scans the source and preserves more of its long-term development. A fast scan can pass across attacks and silences before enough grains have sampled them.

The current source read heads wrap around the source buffer. Although `loop_source` appears in the schema, the granular CLI does not currently apply it to stop wrapping. To make a deliberate finite gesture, render to a chosen length and apply the final fade in an editor or use an

amplitude curve.

Use nonnegative positions in the working range of your file. The current implementation is designed around a simple source-index mapping, not a full transport with documented reverse and boundary modes.

Amplitude belongs to the cloud

Amplitude scales the gains of each emitted grain. It is not a final master limiter. Overlapping grains and multiple clouds add at the output. Similar grains can add coherently, so doubling grain count does not imply a fixed three-decibel change.

A useful beginner range is 0.05 to 0.2 for a modest cloud, followed by a short level check. These are starting values, not guaranteed output peaks. Reduce amplitude before increasing density into the hundreds. Float output retains values beyond nominal full scale, but a player or later conversion may clip them.

Capacity and scheduling

Each cloud uses a particle pool. The current CLI uses `cloud.max_particles` when provided and otherwise constructs a pool of 1,024 grains. The root `engine.max_particles_per_cloud` declaration is not currently propagated as the default pool size by this CLI path. Set a per-cloud limit when capacity matters.

If every particle is active, a new grain cannot be acquired. The result can sound thinner or less predictable than a calculation based only on density suggests. Leave headroom above density multiplied by duration, then test the actual patch.

The current granular engine schedules updates over a block and then processes the pool. Do not assume sample-exact onset placement or identical output when changing block size. Keep block size fixed during comparisons and performance qualification.

Exercise: separate three causes of roughness

Make three rough textures: one with sparse grains, one with very short grains, and one with abrupt window edges. Listen for gaps, spectral broadening, and clicks respectively. Write down which parameter caused each behavior. This distinction makes later troubleshooting much faster.

5. Dronoify: a source becomes an atmosphere

Generate an editable instrument

Dronoify is a patch generator. It takes a WAV path and writes three cooperating clouds: foundation, halo, and sparkle. It does not overwrite or process the source file when generating YAML. Render the generated patch in a second step.

```
particelle preset dronoify audio/music_example.wav \
  --amplitude 0.06 > drone.yaml
particelle validate drone.yaml
particelle render drone.yaml -o renders/drone.wav \
  --duration 30 --format pcm24
```

The reduced amplitude in this example leaves more room for long overlapping grains. It is intentional. The generator's default is 0.32, and the result still depends on the source and the other parameters. Render a short preview before making a long bed.

Understand the three layers

The foundation receives your primary density, duration, amplitude, position, and width. The halo uses 55 percent of the density, 1.75 times the duration, and 55 percent of the amplitude. It starts from a source region offset by 0.22 seconds. The sparkle layer uses 28 percent of the density, 55 percent of the duration, and 32 percent of the amplitude, with a 0.51-second offset.

The layers also differ in scan and orientation rates. These differences let a steady source generate slow internal movement. The supplied oscillators are unipolar, so the source drift expressions add a positive excursion to their base positions. They do not alternate equally above and below the stated position.

Option	Default	What to listen for
`--density`	32	Density of the main layer
`--grain-duration`	0.85 s	Sustain, overlap, and source recognizability
`--amplitude`	0.32	Per-grain level before summing
`--position`	0.5 s	Which source region establishes the drone
`--width`	0.92	Spread in the default panner
`--movement-hz`	0.012	Slow orientation/directivity variation

Option	Default	What to listen for
`--drift-hz`	0.021	Slow movement through source material
`--window`	blackman_harris	Grain-edge shape
`--channels`	2	Output layout size, from 1 to 256

Three useful source choices

For a vocal atmosphere, select a sustained vowel with little room noise. Start just after the consonant, use long grains, and let the halo carry slow change. If every grain repeats the consonant, move position later rather than trying to smooth it away with more overlap.

For a metallic drone, choose a struck resonant object and scan the decay. The early region produces more attack and brightness; the later region emphasizes a few persistent resonances. A very short source may reveal obvious wraparound repetition. Prepare a longer source when that repetition conflicts with the musical goal.

For an environmental bed, choose a recording with stable background activity. Gusts, passing engines, or isolated calls can become recurring motifs. Decide whether those motifs are welcome before increasing grain length: longer grains may preserve the events more clearly.

Multichannel Dronoify

```
particelle preset dronoify audio/music_example.wav \
  --channels 8 --density 24 --grain-duration 1.2 \
  --amplitude 0.04 > drone-eight.yaml
particelle render drone-eight.yaml -o renders/drone-eight.wav \
  --duration 20 --format pcm24
```

This creates eight channels in a horizontal ring. The current amplitude panner only excites speakers in the source-facing hemisphere. A large width does not guarantee energy in every channel of a ring. Review Chapter 8 before promising an all-speaker installation or a height-channel delivery.

Exercise: preserve a trace

Render three thirty-second drones from the same source. In the first, make the source unmistakable. In the second, retain only one recognizable clue. In the third, aim for an abstract sustained texture. Record the source position, duration, and density for each, then explain which changes did the most work.

6. Signals, curves, and controlled movement

A parameter can be a small expression

A signal-valued field accepts a number, a named reference, or an expression with `op` and `args`. A number stays constant. A file path refers to a JSON curve. A string beginning with `$` reads a named control field, except for recognized `$analysis.` references.

The simplest useful animated density is a baseline plus a scaled oscillator. The following complete patch varies density between 8 and 32 grains per second, while the source position scans through the first second every eight seconds.

examples/userguide/moving.yaml

```
schema_version: 2
engine: { sample_rate: 48000, block_size: 256 }
layout:
  channels:
    - { name: L, azimuth_deg: -30.0 }
    - { name: R, azimuth_deg: 30.0 }
clouds:
  - id: moving
    source: audio/music_example.wav
    density:
      op: sum
      args: [8.0, { op: mul, args: [24.0,
        { op: osc, args: [sine, 0.2] }] }]
    duration: 0.12
    position: { op: osc, args: [phasor, 0.125] }
    amplitude: 0.06
    width: 0.6
    listener_pos: { x: 0.0, y: 1.0, z: 0.0 }
    window: { type: hann }
```

A frequency of 0.2 Hz gives a five-second cycle. Sine, triangle, saw, square, and phasor oscillators produce values between zero and one at ordinary positive rates. To obtain a range from minimum to maximum, multiply by the range and add the minimum. For a bipolar range from -1 to 1, multiply by 2 and add -1.

Operator	Arguments	Use
`sum` or `add`	two signals	Add a baseline or combine modulation
`mul` or `multiply`	two signals	Set depth or gate a signal
`clamp`	signal, constant min, constant max	Restrict an input range

Operator	Arguments	Use
`map`	signal, function name	Apply a conversion such as `db_to_linear`
`osc`	shape, frequency, optional phase	Repeating motion; phase is in cycles

Use constant oscillator frequency for a predictable cycle. The current oscillator computes phase from absolute time multiplied by the current rate; changing rate is not the same as integrating a continuously varying frequency. Abrupt rate changes may produce discontinuities.

Draw a phrase with a curve

A curve specifies time/value endpoints and an interpolation shape. In this example, source position travels from zero to two seconds over eight seconds of output. Save the supplied JSON as `examples/userguide/scan.json`.

examples/userguide/scan.json

```
{
  "segments": [
    { "x": 0.0, "y": 0.0,
      "x_end": 8.0, "y_end": 2.0,
      "shape": "linear" }
  ],
  "extrapolation": { "left": "clamp", "right": "clamp" }
}
```

Use it in a complete patch:

examples/userguide/scan.yaml

```
schema_version: 2
engine: { sample_rate: 48000, block_size: 256 }
layout:
  channels:
    - { name: L, azimuth_deg: -30.0 }
    - { name: R, azimuth_deg: 30.0 }
clouds:
  - id: scan
    source: audio/music_example.wav
    density: 24.0
    duration: 0.10
    position: examples/userguide/scan.json
    amplitude: 0.08
    width: 0.6
    listener_pos: { x: 0.0, y: 1.0, z: 0.0 }
    window: { type: hann }

particelle curve examples/userguide/scan.json \
  --resolution 17 > renders/scan.tsv
particelle render examples/userguide/scan.yaml \
  -o renders/scan.wav --duration 8 --format pcm24
```

The phrase samples source positions at a quarter of normal forward progression. It is a granular time-expansion technique, not a promise of phase-coherent time stretching. Individual grains still read forward, and overlaps create their own coloration.

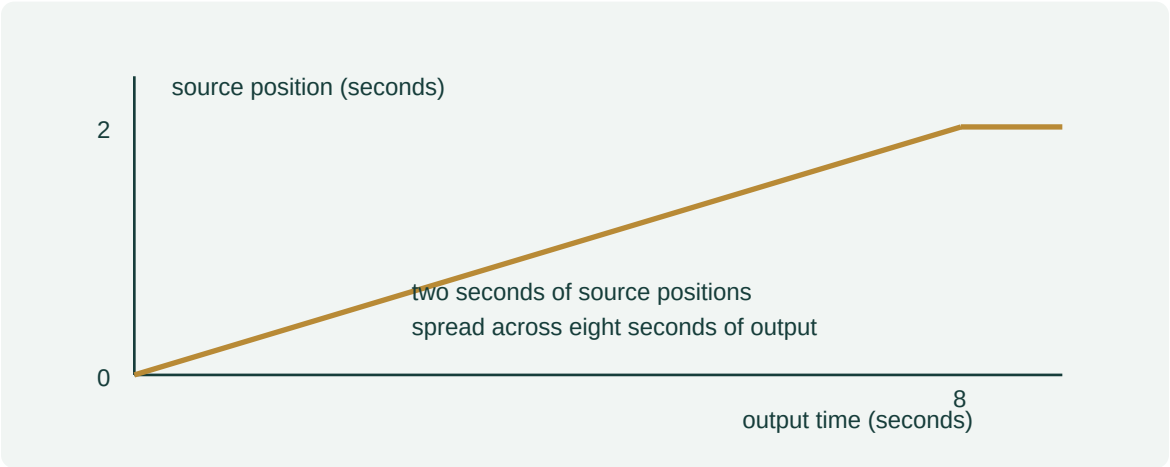


Figure 3. The teaching scan curve reaches two seconds of source position at eight seconds of output, then clamps.

Extrapolation defines the ending

Clamp holds the boundary value; repeat returns to the start; mirror travels back through the curve; zero returns zero outside the domain; linear extends the boundary trend. Select these deliberately. A source-position curve that clamps will keep revisiting the endpoint. An amplitude curve that clamps at a nonzero value will sustain, even if you imagined the phrase was over.

Linear is easy to inspect. Smoothstep and smootherstep soften transitions. More elaborate interpolation shapes belong after you can predict the result of a linear segment. Check the TSV output when the shape does not match your expectation.

Exercise: design a beginning and an ending

Make a sixteen-second study with three stages: sparse opening, dense middle, and a fading end. Use separate curves for density and amplitude. Explain why setting density to zero stops new grains but does not instantly remove already active grains. Render beyond the nominal ending to inspect the tail.

7. Let the audio control the patch

Analysis is a prepared control signal

An analysis block reads a source file, extracts a feature, and gives the result a name. A cloud can refer to it through `$analysis.name`. The current CLI prepares these vectors offline before synthesis; this is not live microphone analysis.

The analysis path reads the first source channel. If your meaningful signal is only on the right, prepare a mono analysis file or reorganize the source. Tracking is interpreted at approximately 100 values per second, then interpolated while the engine runs.

A feature is measured in its own units. RMS is an amplitude measure, a spectral centroid is a frequency-like measure, and loudness in dBFS is logarithmic. Connect them through a scale and a bound that makes sense for the destination. Raw centroid values make poor grain densities without conversion.

A level-following density patch

examples/userguide/follower.yaml

```
schema_version: 2
engine: { sample_rate: 48000, block_size: 256 }
layout:
  channels:
    - { name: L, azimuth_deg: -30.0 }
    - { name: R, azimuth_deg: 30.0 }
analysis:
  - id: level
    source: audio/music_example.wav
    extractor: rms
clouds:
  - id: follower
    source: audio/music_example.wav
    density:
      op: clamp
      args:
        - op: sum
          args: [8.0, { op: mul,
            args: [80.0, "$analysis.level"] }]
        - 8.0
        - 48.0
    duration: 0.1
    position: { op: osc, args: [phasor, 0.1] }
    amplitude: 0.05
    width: 0.6
    listener_pos: { x: 0.0, y: 1.0, z: 0.0 }
    window: { type: hann }
```

The baseline of eight grains per second keeps the cloud alive in quieter passages. The scale of eighty makes the source level influential, while the clamp limits the operating range. Render

this beside a constant-density version. The meaningful question is whether the extracted contour makes the gesture more readable, not whether a complex expression exists.

Select a feature for a musical reason

Feature family	Examples accepted by the CLI	Possible mapping
Level	`rms`, `peak_amplitude`, `loudness_dbfs`	Density or amplitude
Brightness	`spectral_centroid`, `spectral_rolloff`	Grain duration or source position
Change	`spectral_flux`	Accent density or motion depth
Noise/tonality	`spectral_flatness`, `harmonic_ratio`	Width or texture balance
Pitch	`f0_yin`, `chroma_strength`, `chroma_C`	Control contours; see tuning limits

Treat noisy or unreliable feature values as an expected design issue. A pitch tracker can become unstable on percussion; a low-level recording can produce an almost flat envelope. Inspect the source and use a baseline, range limit, or prepared control curve when necessary.

When a render outlasts the analysis vector, the current parameter evaluator holds its final value. It does not loop the analysis simply because the audio read heads wrap. Long-form textures need an explicit plan for that sustained control value.

Exercise: one source conducts another

Use a rhythmic source for analysis and a sustained source for the cloud. Map the rhythmic source's RMS to density, then to amplitude in a second version. The first changes the arrival of material; the second changes its level. Compare the residual tail after each accent and explain why the two versions differ.

8. Compose beyond stereo

A layout is an ordered list

Output channel order follows `layout.channels`. Speaker names are useful documentation, but they do not guarantee how a DAW or player assigns a multichannel WAV. Write down your channel map and test it on the receiving system.

Spherical positions use azimuth and elevation in degrees. Zero azimuth points forward; positive azimuth turns toward the right; positive elevation points upward. Cartesian coordinates use positive x to the right, positive y forward, and positive z upward. The panner normalizes direction, so these coordinates should not be interpreted as a complete distance/acoustics model.

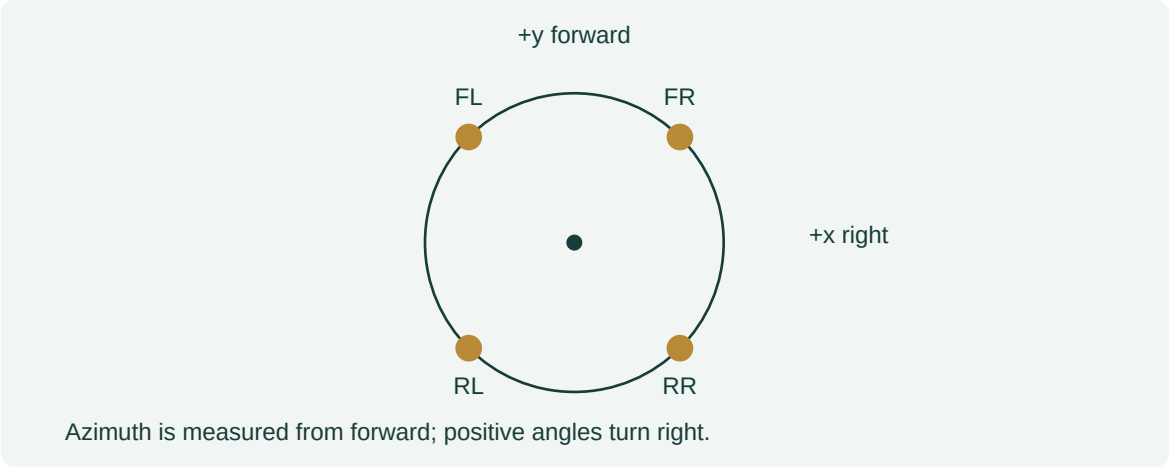


Figure 4. The four-channel teaching layout viewed from above. Channel order is FL, FR, RL, RR.

A four-channel teaching patch

The following patch uses opposing front and rear clouds so the default panner has material in both hemispheres. A single forward-facing cloud with a large width would not necessarily energize the rear pair.

examples/userguide/quad.yaml

```
schema_version: 2
engine: { sample_rate: 48000, block_size: 256 }
layout:
  channels:
    - { name: FL, azimuth_deg: -45.0 }
    - { name: FR, azimuth_deg: 45.0 }
    - { name: RL, azimuth_deg: -135.0 }
    - { name: RR, azimuth_deg: 135.0 }
clouds:
  - id: front
    source: audio/music_example.wav
    density: 16.0
    duration: 0.15
    position: 0.5
    amplitude: 0.06
    width: 0.8
    listener_pos: { x: 0.0, y: 1.0, z: 0.0 }
    window: { type: hann }
  - id: rear
    source: audio/music_example.wav
    density: 11.0
    duration: 0.22
    position: 1.0
    amplitude: 0.04
    width: 0.8
    listener_pos: { x: 0.0, y: -1.0, z: 0.0 }
    window: { type: hann }

particelle render examples/userguide/quad.yaml \
  -o renders/quad.wav --duration 8 --format pcm24
```

Import it into a four-channel track or route its four channels explicitly. A stereo player may downmix, ignore channels, or refuse the file. That behavior says little about whether the file was rendered correctly.

Width, direction, and radiation

The default amplitude panner favors speakers whose direction has a positive dot product with the source direction. Width changes how concentrated the gains are within that hemisphere. Width near one flattens that distribution, but it does not turn rear-facing speakers into front-facing ones.

`orientation_azimuth`, `orientation_elevation`, and `directivity` alter the source's radiation gain relative to the listener. Rotating orientation can therefore change level without moving a point around the loudspeaker ring. To design a clear front/rear gesture, begin with separate clouds at different `listener_pos` values, then shape their amplitudes.

Use the channel count generated by `init -n` as a starting layout only. Above two channels the generator creates a ring at zero elevation. If you need height speakers, explicitly edit their elevation. A channel called LFE receives ordinary synthesized audio unless you build a separate

bass-management process; its name does not create a low-pass filter.

Delivery is another stage

Particelle produces channel-based WAVs. It does not author Dolby Atmos objects, ADM metadata, or a finished immersive distribution package. Prepare such delivery in an appropriate downstream authoring workflow, using a verified channel map.

The Mo adapter's multichannel method is different again: stereo is preserved at two channels; larger buses receive copies of a normalized mono field. This ensures a usable bus but does not supply independently moving spatial voices. Do not interchange those two spatial behaviors in a production specification.

Exercise: verify the room before composing it

Render the front cloud alone and confirm that it reaches FL and FR. Render the rear cloud alone and confirm RL and RR. Then combine them. Write the routing map beside the patch so the study can be reconstructed on another interface.

9. Tuning and pitch: use the connected path

Three different meanings of pitch control

Pitch can mean a source recording's original note, a grain's playback speed, or the note assigned to a synthesizer voice. These are related but distinct. Slowing sample playback changes duration and spectrum together. Moving a grain's start position changes which source material is selected, but does not itself transpose that material.

The repository contains tuning libraries for twelve-tone equal temperament, arbitrary equal divisions of the octave, just-intonation ratios, and Scala scales. The YAML schema accepts tuning selections. However, the current granular `build_engine` path does not apply that tuning selection to cloud playback rate. It also does not compile a `playback_rate` field from the cloud YAML schema.

Consequently, adding `tuning: { mode: edo, steps: 31 }` to a granular patch is not enough to hear 31-EDO transposition. Some older examples contain pitch-oriented fields that the current parser can ignore. Use a short audible test before attributing a pitch change to such a field.

What an interval calculation tells you

In an N-division equal temperament, k steps represent a frequency ratio of 2 raised to k/N. Twelve steps in 12-EDO give a ratio of two. A just-intonation fifth represented as 3/2 gives a ratio of 1.5. These calculations describe desired frequencies; an audio path still has to apply them.

For granular work today, prepare source files at the intended pitches or use the Rust engine API where playback-rate signals can be assigned programmatically. Keep source rate and output rate consistent when judging pitch.

Scala and keyboard maps in Mo

Obelisk Mo's render and live binaries load Scala tuning through `--scl` and optional `--kbm`. A scale describes interval structure; a keyboard map describes how MIDI keys correspond to it, including reference pitch and unmapped notes. An unmapped note should not consume an audible voice.

The following command is a template: replace the two filenames with your own scale and map.

```
cargo run --release -p obelisk-m0 --bin obelisk-m0-render -- \
  --scl my-scale.scl --kbm my-map.kbm \
  --output renders/tuned-m0.wav --duration 10
```

Test the reference note, an octave-related note, and at least one unmapped key before a performance. Check that the controller and the instrument agree on pitch-bend range. A correct static tuning can still sound wrong if per-note bend is scaled differently at the controller and synthesizer.

Exercise: write a tuning verification plan

Choose a scale and list three exact target frequencies or ratios. Record which keys should play them and which keys should remain silent. Render or play those notes in M0, then compare against your expectations. Keep this verification separate from judging whether the scale makes a compelling musical phrase.

10. Realtime, MIDI, and OSC

Establish the device path first

Begin with the basic stereo patch. Confirm offline playback, then run it for a bounded duration and save diagnostics.

```
particelle run examples/userguide/basic.yaml \
  --duration 20 --telemetry-file renders/realtime.json
```

The audio device must support the configured sample rate and output-channel count. `hardware.device_name` can select a device; absent a name, the default output is used. A patch that renders to twelve channels offline may fail on a two-channel device.

A 256-frame block at 48 kHz lasts about 5.33 milliseconds. That is a processing deadline, not a complete round-trip latency measurement. Device buffers, operating-system scheduling, and the rest of a performance rig add latency of their own.

The granular realtime path includes control-thread synchronization and per-grain work that is not covered by M0's zero-allocation audit. Use the actual interface and workload when judging readiness. M0 audit results do not qualify the separate granular CLI.

Route a control field into a sound

MIDI routing writes named fields. A patch must reference those fields to make them audible. The following patch maps modulation wheel CC1 to `energy`, then scales that field into density. A baseline keeps the cloud audible before the first controller message arrives.

examples/userguide/control.yaml

```
schema_version: 2
engine: { sample_rate: 48000, block_size: 256 }
layout:
  channels:
    - { name: L, azimuth_deg: -30.0 }
    - { name: R, azimuth_deg: 30.0 }
routing:
  midi_bindings:
    - { source: midi.cc.1, target: energy }
clouds:
  - id: control
    source: audio/music_example.wav
    density:
      op: sum
      args: [8.0, { op: mul, args: [32.0,
        { op: clamp, args: ["$energy", 0.0, 1.0] }] }]
    duration: 0.12
    position: 0.5
    amplitude: 0.06
    width: 0.6
    listener_pos: { x: 0.0, y: 1.0, z: 0.0 }
    window: { type: hann }
```

Select a MIDI port by its exact name using `--midi-port`. The patch's `hardware.midi_input` provides another configuration route. `--simulate-mpe` supplies a test stream; it is not evidence that a physical controller is connected.

The current CLI constructs direct routing rules and does not apply the optional binding `transform` expression. Put scaling and clamping in the destination signal graph, as the example does. Unbound fields evaluate to zero. The router's `midi.note` value is note velocity, not MIDI note number.

Speak OSC

```
particelle run examples/userguide/control.yaml \
  --osc-port 9000 --osc-reply --duration 30
```

Use an OSC-capable sender to send `/field/energy` with a numeric argument between zero and one. This changes the same field that the MIDI mapping writes. The CLI understands binary OSC packets over UDP; sending a plain text string with a generic network utility is not equivalent.

Request	Meaning	Reply when enabled
`/ping`	Check the responder	`/pong` with string `particelle`
`/field/energy 0.75`	Set field `energy`	`/ack` with field name and numeric value

Request	Meaning	Reply when enabled
Unsupported address	No recognized operation	`/error` with explanatory text
Field without a numeric argument	Invalid update	`/error` with explanatory text

Replies go back to the sender. An acknowledgement confirms handling of the control message, not that sound was heard or a target field was connected. OSC bundles are traversed in order; the current handler does not schedule them from their timetags. Several writes to one field in a control drain use the last value. OSC updates take precedence when MIDI and OSC target the same field in that drain cycle.

Rehearse failure as well as success

A bounded normal exit produces the telemetry JSON. Do not rely on an interrupted process to produce the same final report. Inspect deadline misses and dropped callbacks alongside your listening notes. If the patch struggles, reduce density or duration, simplify the analysis/control workload, or test a larger block with the resulting musical tradeoff in mind.

The patch watcher can rebuild an engine after edits. Do not assume continuity of grain state or click-free changes during a performance. Rehearse the exact editing process if live patch changes are part of the work.

Exercise: one gesture, two controllers

Control density first from a MIDI wheel and then from OSC. Keep the same field and output range. Confirm that the baseline produces sound with neither controller active, that both paths reach the same maximum, and that an OSC ping/reply works independently of the audio mapping.

11. Obelisk M0 for performers and host developers

A different source of sound

M0 combines frequency-aware wavetable excitation with a sixteen-mode resonant body, per-note expression, filtering, and release behavior. Pressure influences brightness; timbre/CC74 influences coupling and morph behavior. Its audio samples are f32, with f64 used in tuning and other calculations. Do not apply the granular engine's f64 description indiscriminately to M0.

Render the built-in performance before connecting a controller:

```
cargo run --release -p obelisk-m0 --bin obelisk-m0-render -- \
  --output renders/m0.wav --duration 5
```

The renderer reports peak level, a sample-data hash, callback timing ratios, and audited audio-thread allocations. Its WAV output is 32-bit float. When a player has float-WAV compatibility trouble, convert it in an audio editor or SoX to PCM24.

Play a controller

```
cargo run --release -p obelisk-m0 --bin obelisk-m0-live -- --list
cargo run --release -p obelisk-m0 --bin obelisk-m0-live
```

Select named devices with `--midi` and `--audio` when needed. The live path uses a bounded MIDI queue and has overflow handling intended to prevent a dropped note-off from leaving a permanent note. A useful physical test includes pressure before note-on, independent bends on held notes, channel reuse, release, and reconnection.

Start with a small voice count or a modest playing pattern while establishing device settings. The standalone stress renderer is a useful software check, but physical-controller and DAW qualification require the actual equipment.

Preset state and host state

Factory presets live in `presets/obelisk-m0/`. They use a versioned JSON schema with `base_morph`, `modulation_hz`, `delay_mix`, and `reverb_mix`. They are library-facing state in this edition; the standalone render/live parsers do not expose a preset-loading command.

The host adapter also has versioned session JSON for the four controls. A factory preset and a host session are separate formats. The host uses `save_state` and `restore_state` with exclusive access on the control thread. Parsing or validation failures leave controls unchanged. State

operations allocate and must not run in an audio callback.

Session state intentionally excludes active voices, effect tails, bus configuration, and external tuning. The host configures these at their own lifecycle points. `all_notes_off` releases held voices at a block boundary; envelopes and tails continue to decay rather than disappearing instantly.

Integration boundaries

`M0HostAdapter` accepts block-boundary control changes, sample-offset MIDI events, planar output, and host frequency tables. The C API provides opaque handles and stereo processing. The wrapper remains responsible for host lifecycle, UI, parameter identification, format-specific state exchange, and external tuning discovery.

The external tuning bridge accepts 128 positive frequencies. The current engine applies its cached base frequencies when voices start; do not assume that replacing the table retunes already sounding voices. Test that distinction if continuous retuning is central to a piece.

For more than two adapter output channels, the current M0 diffuse output distributes a normalized mono field. It is useful for compatibility but is not a spatial decorrelator. The simple feedback-based ambience controls are also a foundation, not a measured-room model.

Exercise: qualify one expressive phrase

Create a phrase with two held notes, independent pitch bends, different pressures, a timbre sweep, and overlapping releases. Save the setup and replay it. Listen specifically for expression leaking from one note to another or a release changing when a channel is reassigned. Attach your controller model and device settings to the results.

12. A practical sound-design cookbook

Freeze a recognizable moment

Use `basic.yaml` as the starting point. Select an interesting source position, raise duration to 0.3 seconds, set density near 24, and lower amplitude to 0.04 for the first preview. The grains keep revisiting the chosen region, creating a stationary view of the source.

If the attack dominates, move the position later. If the sound becomes too periodic, introduce a small position scan with a low-rate oscillator. If it loses identity, shorten duration or reduce position movement before increasing level.

Listening target: the sound should remain related to the source while losing the source's original timeline. Compare a vowel, a bell, and a burst of noise to hear how strongly the material controls the result.

Make a granular pulse

Begin with density 4 and duration 0.04 seconds, amplitude 0.1, and a Hann window. Use a short, clear source region. Increase density until pulses fuse into a buzz or texture, then return to the point where the rhythmic identity is useful.

For a changing pulse rate, use a curve so you can inspect the intended timing contour. This engine's block scheduling is not a sample-accurate drum sequencer; if precise synchronization is essential, render the texture and place the result on the DAW timeline.

Listening target: choose whether the listener should follow individual onsets or hear an aggregate texture. The transition is often more interesting than either extreme.

Stretch a gesture

Use the supplied scan curve, which moves through two seconds of source positions during eight seconds of output. Increase density if holes become distracting, and adjust grain length to preserve or dissolve attacks. Keep source and output sample rates matched.

A slow scan and a long grain are not equivalent. Scan determines where new material enters; duration determines how long old material remains. With very long grains, later regions overlap earlier ones and the gesture can become a harmonic mixture instead of a prolonged phrase.

Listening target: preserve the overall contour while altering the apparent duration. Describe which attacks survive and which smear into the surrounding texture.

Build a front/rear question and answer

Start with `quad.yaml`. Put an amplitude curve on the front cloud that rises during the first half of the phrase and falls during the second. Give the rear cloud the complementary contour. Keep total level moderate during the crossover.

Because the source regions and densities differ, the rear answer can sound like a transformation rather than a moved copy. For an unambiguous routing test, use one cloud at a time before auditioning the musical overlap.

Listening target: a listener should hear both a spatial exchange and a relationship between the two materials. If the relationship disappears, first reduce the differences in source position.

Create a self-following atmosphere

Use `follower.yaml`, then lengthen the grains and lower their amplitude. The source's level contour changes density while the scan moves through its internal material. Compare the result against Dronoify to hear the difference between externally measured motion and periodic internal motion.

Try analyzing a different file while keeping the cloud source unchanged. This separates the gestural identity of one performance from the timbral identity of another. Preserve both files in the project archive because both determine the render.

Finish a study as a deliverable

A useful small composition can have an opening, a sustained development, and an ending without needing a large number of clouds. Commit to a duration, render short sections during development, then render the complete file. Add the final edit and fade in your delivery workflow if the patch does not provide them.

For each study, save a brief listening note: what changed, what remained identifiable, and which parameter mattered most. Over several studies, this becomes a personal cookbook tied to audible evidence rather than a collection of impressive-looking patches.

13. Windows and the shape of a grain

Smooth edges change the spectrum

A grain window multiplies the selected source excerpt. Abruptly turning a waveform on or off introduces sharp changes that can be heard as clicks or broadband energy. A smooth window reduces those edges and changes how overlapping grains add.

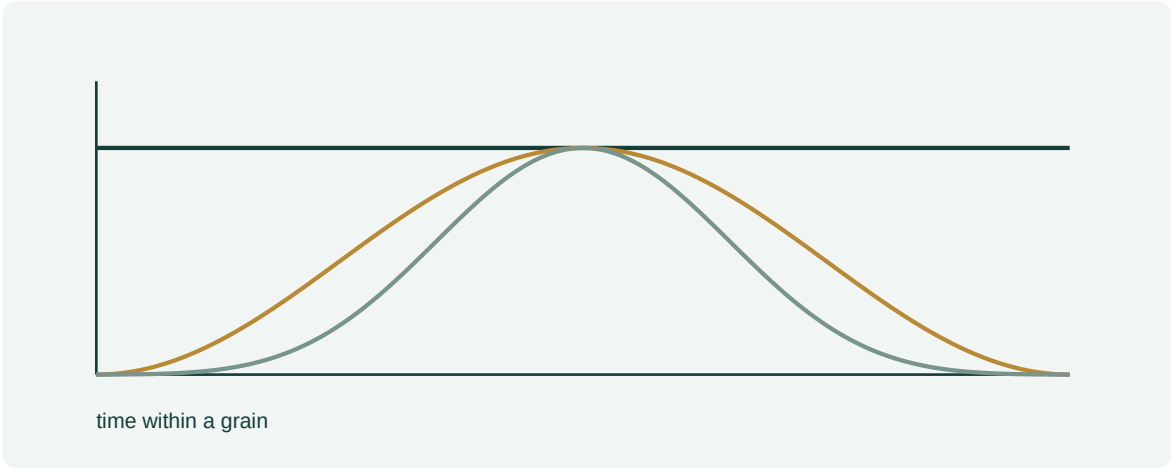


Figure 5. Rectangular (green), Hann (gold), and Blackman-Harris (gray-green) windows at equal peak height.

Hann is a good starting point because it has smooth endpoints and a clear single-lobed shape. Rectangular keeps full level until the boundary, useful for demonstrating edge effects but often too abrupt for a smooth cloud. Blackman-Harris emphasizes a smoothly tapered center and is the Dronoify default.

The window is not a filter that simply removes a fixed frequency band. Window duration, source content, and grain overlap interact. Shortening a window broadens the spectral influence of the time-localized excerpt; increasing overlap can smooth the amplitude contour without recreating the original source exactly.

Choose by listening, then refine

Starting choice	Character to investigate	Good first comparison
Hann	General smooth grain envelope	Basic patch at several durations
Rectangular	Abrupt boundaries	Same patch at low density
Blackman-Harris	Strongly tapered grain	Long-grain drone
Triangle/Bartlett	Linear rise and fall	Sparse rhythmic texture
Kaiser	Adjustable shape	Controlled comparison with explicit parameters

The public window schema uses `bartlett` for the triangle family. Do not invent a `triangle` window token merely because the oscillator supports a triangle waveform; these are different namespaces. Additional windows and parameterized forms are defined in `particelle-dsp/src/window/schema.rs`.

The CLI builds a peak-normalized window lookup table. Peak normalization keeps the largest window value comparable; it does not equalize integrated energy across all windows. Loudness-match window comparisons before deciding which one is brighter or fuller.

Exercise: hear a boundary

Use one sustained source region and a low grain density. Render Hann, rectangular, and Blackman-Harris versions with identical duration and amplitude. Listen for edge clicks, the apparent length of the grain, and changes in level. Then increase density and compare again; overlap can hide an edge that was obvious in isolation.

14. Rendering, quality control, and archiving

Choose an encoding for the next stage

The default float format is convenient for further processing. PCM24 is a practical sharing format for players and audio tools that are less tolerant of float-WAV headers. PCM16 is useful when a specific destination requires it. The `--pcm24` flag is a shortcut and takes precedence over the ordinary format selection.

```
particelle render examples/userguide/basic.yaml \
  -o renders/archive.wav --duration 10 --format f32 --hash
particelle render examples/userguide/basic.yaml \
  -o renders/share.wav --duration 10 --format pcm24
```

The CLI does not expose a 64-bit-float WAV output option, even though the granular calculations use f64 internally. Internal precision and delivery encoding are separate choices. Do not describe an ordinary default render as a 64-bit WAV.

Inspect level and channel activity

For a short render, confirm sample rate, frame count, channel count, and peak. If using SoX, `soxi renders/share.wav` reports format information and `sox renders/share.wav -n stats` reports level statistics. Listen to the result as well; measurements do not identify a musically unwanted loop or a badly chosen source segment.

A float peak above 1.0 exceeds nominal full scale. Lower patch amplitudes and rerender before producing an integer deliverable. The renderer is not a mastering chain with automatic loudness targeting or a guaranteed final limiter. Dense correlated grains can reach high peaks even if each amplitude number looks small.

For multichannel work, inspect each channel individually. Silence can be intentional under the hemisphere-based panner. If a delivery requires every channel active, demonstrate that requirement with the actual patch rather than inferring it from channel count.

Reproduce the result you actually tested

Keep the executable version or commit, patch, source WAVs, curves, analysis files, duration, sample rate, block size, and output encoding. Repeat a render with exactly those inputs before claiming repeatability.

The granular CLI's `--hash` hashes the complete output file. Mo reports a hash of its rendered sample values. Those hashes are not interchangeable. Across architectures, library versions, or

changed blocks, do not promise an identical digest without running the relevant comparison. The repository's render-parity scripts are the place to check its narrower tested policy.

The renderer holds source buffers in memory. Large files, many clouds, and large particle pools increase memory use. Estimate capacity and try a short render before asking for a long multichannel artifact.

A handoff checklist

- Give the audio file a descriptive name and state its channel order.
- Include the patch and all assets required to recreate it.
- Record source and output rates and any conversion performed beforehand.
- Note whether final fades, normalization, or other edits happened afterward.
- Preserve one short preview that can be heard without a specialist playback rig.

Exercise: a reproducibility exchange

Give a collaborator the project folder and one exact render command. Ask them to reproduce the WAV without editing the patch. Compare format information and hashes within the expected build environment, then compare listening impressions. Fix missing paths or ambiguous instructions before adding more features.

15. Troubleshooting by symptom

The shell complains about a brace or a comment

A message such as `zsh: parse error near '}'` usually means YAML was pasted into the terminal. Put it in a `.yaml` file and give that filename to `validate` or `render`. A `zsh: command not found: #` message may come from copying explanatory comment lines into a shell that does not enable interactive comments. This book keeps explanation outside executable shell blocks.

The source cannot be opened

Check the current directory, then the path in the patch. `audio/music_example.wav` is relative to where the CLI is launched. Moving only the YAML file does not move its source. A valid schema does not establish that the WAV exists, that it can be decoded, or that it contains sound.

If the source is MP3, AAC, or another unsupported format, convert it to WAV with a tool you trust. Match its sample rate to the patch's engine rate. Keep the converted file as an explicit project asset.

The render exists but is silent

Listen to the source. Then return to the basic patch's constant values: positive density, positive duration, moderate amplitude, a Hann window, and a forward listener position. Temporarily remove field references and analysis mappings. An uninitialized `$field` evaluates to zero and can mute amplitude or stop emission.

Check spatial geometry next. A source facing away from all speakers can produce zero gains. Directivity can also reduce radiation to zero. Compare with a stereo layout and `directivity: 1.0`, which avoids orientation attenuation in the current implementation.

A control seems to do nothing

Confirm that the field name written by MIDI or OSC exactly matches the `$field` used in the cloud. The target is a field name, not an automatic path into the YAML tree. Sending a message to a cloud-like path does not alter density unless the density expression reads that field.

If the control concerns tuning, playback rate, loop behavior, or a binding transform, consult the capability notes. The current parser can accept or ignore fields without producing the implied audio behavior. Avoid diagnosing an unconnected feature by repeatedly increasing its value.

A patch becomes invalid after using set

The current `set` command is a text replacement based on the last segment of the supplied path. It can replace every line with the same key, and it does not understand nested signal expressions. For example, a density replacement can affect several clouds or leave a stale indented expression body.

Use `set` only for an unambiguous scalar line, such as a single `engine.block_size`. Write the result to a new filename, inspect it, and validate it. Edit structured or repeated fields with a YAML-aware editor. Never redirect `set patch.yaml ...` back onto `patch.yaml`; shell redirection can truncate the input before it is read.

Playback reports a WAV header warning

Render PCM24 for a compatibility copy. The message `wave header missing extended part of fmt chunk` has appeared with some float-WAV playback paths. A successful PCM24 render is a useful way to distinguish player/container compatibility from synthesis failure.

Realtime clicks or drops out

Render offline to determine whether the click is in the patch or the device path. If it is present offline, inspect windows, discontinuous curves, source boundaries, and clipping. If it appears only live, inspect telemetry and test a lower processing load on the same device.

A larger block changes the callback budget and may change the current granular scheduling result. Treat it as a new render/performance configuration, and record it in the project notes. A device soak with the real patch is more useful than a generic statement that the engine is realtime capable.

16. Command and patch reference

Command map

Command	Essential input	Typical result
`init`	Optional `--channels` / `-n`	Starter YAML on stdout
`preset dronoify`	Source WAV and options	Layered YAML on stdout
`validate`	Patch filename or `-`	Schema report
`render`	Patch, `-o`, optional duration/format	Offline WAV
`run`	Patch, optional devices/control flags	Realtime output
`curve`	JSON file, optional resolution	TSV time/value samples
`set`	Patch, parameter name, value	Text-modified YAML on stdout

Use `particelle --help` and `particelle COMMAND --help` to inspect the installed binary. This book's behavior notes take precedence over examples in old help text when discussing the documented source snapshot. Validate and preview after any change.

Root patch blocks

Block	Purpose	Important detail
`schema_version`	Compatibility version	Current generated patches use 2
`engine`	Rate and processing block	Keep rate matched to source
`layout`	Ordered output channels	Generator counts are 1 through 256
`clouds`	One or more sound layers	Each loads its own source path
`analysis`	Offline feature vectors	Named by `id` and referenced with `$analysis.`
`routing`	MIDI field assignments	Scaling belongs in the signal expression
`hardware`	Device and MIDI selection	Realtime path only
`tuning`	Tuning declaration	Granular CLI connection is incomplete

Cloud controls to reach for first

Field	Meaning	Practical constraint
`id`	Layer name	Use stable, readable names
`source`	WAV path	Resolve from working directory
`density`	Grains per second	Positive for continuing emission
`duration`	Grain length in seconds	Positive; affects overlap
`position`	Source read start in seconds	Use matching source/output rates
`amplitude`	Per-grain scale	Summed output can exceed full scale
`window`	Grain envelope	Start with `{ type: hann }`
`width`	Panner spread control	Use 0 through 1 as the working range
`listener_pos`	Direction vector for panning	x right, y forward, z up
`max_particles`	Pool capacity	CLI fallback is 1,024 per cloud

What validation does and does not establish

The schema validator checks selected properties, including positive sample rate, nonzero block size, nonempty layout, known window names, and basic tuning constraints. It is not a complete static analyzer for every expression or asset.

Expression arity and curve decoding are checked later while building the engine. Asset loading can fail after validation. Some unsupported fields may be ignored by deserialization. A complete acceptance sequence therefore includes schema validation, a short render, format/level inspection, and listening.

For programmatic patch generation, use canonical schema version 2, emit only documented fields, and validate your output. AI-generated patches are proposals to inspect, not an alternate implementation of the engine. Preserve the prompt only as supplementary context; the actual YAML and assets determine the render.

17. A short course in granular composition

Session one: one source, one cloud

Work only with the basic patch and a single source. Make a five-second study for each of three densities and each of three durations. Avoid changing both at once. Label the results by parameter values and sort them from most fragmentary to most continuous.

Your written response should identify at least one case where source content mattered more than a numerical setting. A silent source region, a sharp transient, or a nearly periodic vowel can dominate the apparent behavior of the same cloud recipe.

Session two: a phrase instead of a loop

Use curves to write a beginning, development, and ending. Keep the phrase under twenty seconds. Make one version where density shapes the form and another where amplitude shapes it. Explain which version feels more articulated and which feels more continuous.

Your deliverable is the patch, curve files, source, rendered WAV, and a short paragraph describing the intended listening path. A collection of curve points without an audible formal intention is not yet a composition.

Session three: space as structure

Use the four-channel patch. Assign a distinct role to front and rear: statement/answer, stable/unstable, near/far impression, or recognizable/abstract. Use level and material contrast thoughtfully; the current panner does not simulate complete room acoustics or distance cues.

Prepare a channel map and a stereo reference for a listener who lacks the playback array. Explain what the stereo reference preserves and what it loses. Verify channel activity before a rehearsal so routing problems do not consume the compositional session.

Session four: a performed parameter

Map one MIDI or OSC field to a bounded destination. Learn its response well enough to perform a repeatable gesture. Record a version with a controller and a version with a predetermined curve. Compare timing, overshoot, and expressive intent.

If using Mo, make the gesture per-note and test overlapping voices. If using the granular CLI, explain that a shared field affects the cloud rather than creating a separate expressive synthesizer voice for every MIDI note.

Final project: a one-minute work

Make a one-minute composition from no more than two source recordings and four clouds. Include one static region, one evolving scan, one contrast in density, and an explicit ending. A multichannel version is optional, but a reproducible render and a documented channel map are required.

Assess the work by the clarity of its form, the relationship between source and result, control of level and silence, and the quality of its project archive. Complexity is not an assessment criterion. A small patch whose behavior you understand is a stronger foundation than a large patch whose most important fields are unconnected.

18. Glossary and further reading

Working vocabulary

Amplitude. A scale applied to a signal. In a granular cloud it acts on emitted grains; output level also depends on overlap and source correlation.

Block. A group of audio frames processed together. Its duration is block size divided by sample rate.

Cloud. A stream of grains sharing source material and parameter signals.

Density. The intended number of grain onsets per second.

Directivity. Orientation-dependent radiation behavior. In the default panner it can change gain without changing the cloud's position.

EDO. Equal divisions of the octave. An interval of k steps in N-EDO has ratio 2 raised to k/N.

Field. A named runtime control value that a `$name` reference reads.

Frame. One simultaneous sample from every output channel. A stereo frame contains two scalar samples.

Grain. A short, windowed excerpt of a source signal.

KBM. A Scala keyboard map that relates MIDI keys to a scale and reference pitch.

MPE. MIDI Polyphonic Expression: a way to give notes independent expressive control through channel assignments.

OSC. Open Sound Control, a typed message format commonly transported over UDP. It is unrelated to the YAML `osc` operator that generates an oscillator signal.

Overlap. The coexistence of several active grains. Density multiplied by duration gives a useful average estimate.

PCM. Pulse-code modulation. PCM16 and PCM24 identify integer sample encodings of different bit depths.

Position. The source location from which a new grain begins reading.

Schema. The structure and field definitions accepted when a patch or state document is parsed.

SCL. A Scala scale file that describes interval relationships.

Signal graph. A collection of constants, references, and operations evaluated to produce parameter values.

Window. An envelope multiplied with a grain to shape its temporal boundary.

Repository reading map

The following sources underpin this edition. Paths are relative to the repository root and identify the implementation or maintained notes to inspect when behavior changes.

Question	Source
Which CLI flags and expression operators exist?	`particelle-cli/src/main.rs`
Which YAML fields are parsed?	`particelle-schema/src/config.rs`
What does validation actually check?	`particelle-schema/src/validation.rs`
How are grains scheduled and mixed?	`particelle-core/src/engine.rs` and `grain.rs`
How does default panning work?	`particelle-core/src/spatializer.rs`
How are signals and curves evaluated?	`particelle-params/src/signal.rs` and `particelle-curve/`
What are OSC replies?	`particelle-cli/src/osc_control.rs`
How is hardware tested?	`docs/REALTIME_OPERATIONS.md`
What is published in releases?	`docs/RELEASE_ASSURANCE.md`
What does Mo expose to a host?	`docs/OBELISK_M0_API.md` and `obelisk-m0/src/host.rs`
Which host/device claims have evidence?	`docs/OBELISK_M0_QUALIFICATION.md`

Edition and verification notes

This is an authored guide to the current source tree, not a verbatim consolidation of earlier marketing text. Its examples use canonical patch syntax and are shipped alongside the book. The example verification script renders the teaching patches, checks finite nonzero audio, checks expected frame/channel counts, and repeats one render to compare bytes. The PDF build includes a linked contents page, chapter bookmarks, original vector diagrams, and running page furniture.

Hardware playback, physical MPE response, and DAW hosting still require equipment-specific testing. These activities are identified as listening or performance exercises rather than claimed as completed by a documentation build.

The project is maintained at https://github.com/TheColby/Particelle. Consult the repository's LICENSE for distribution terms. All diagrams in this book are generated specifically for the guide.